Access Modifiers in Java

Access modifiers in Java are keywords that define where a class, method, or variable can be accessed. These modifiers help to protect data and control access to code. In this chapter, we will learn about the access modifiers and how they control the visibility of classes, methods, and variables.

- public
- private
- protected
- default

Access Modifier	Within Class	Within Package	Outside Package by Subclass Only	Outside Package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

1. public Access Modifier

The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package. It has the widest scope among all other modifiers.

```
//save by Main.java
package mypack;
import pack.*;
class Main {
  public static void main (String args[]) {
   A obj = new A();
   obj.msg();
  }
}
```

2. private Access Modifier

The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}
public class Main{
 public static void main(String args[]){
  A obj=new A();
  System.out.println(obj.data);//Compile Time Error
  obj.msg();//Compile Time Error
  }
}
```

3. protected Access Modifier

The protected access modifier is accessible within the package and outside the package, but through inheritance only. The protected access modifier can be applied to the data member, method and constructor. It cannot be applied to the class. It provides more accessibility than the default modifier.

```
//save by Main.java
package mypack;
import pack.*;
class Main extends A {
 public static void main (String args[]) {
  B obj = new B();
  obj.msg();
 }
}
```

4. Default Access Modifier

When we do not specify any specifier, the default access specifier is prefixed by default. The default modifier is accessible within a package only. It cannot be accessed from outside the package. It provides more accessibility than private. But it is more restrictive than protected and public.

```java
//save by Main.java
package mypack;
import pack.*;
public class Main {
 public static void main (String args[])  {
  A obj = new A();//Compile Time Error
  obj.msg();        //Compile Time Error
 }
}
```

AccessM.java > AccessM

```java
// Demonstration of Java Basics + Access Modifiers

class Student {

    // Private variable (Encapsulation)
    private String name;

    // Default variable
    int rollNo;

    // Constructor
    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    // Public method
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }

    // Protected method
    protected void checkResult(int marks) {
        if (marks >= 40) {
            System.out.println(x: "Result: Pass");
        } else {
            System.out.println(x: "Result: Fail");
        }
    }

    // Default method
    void showMessage() {
        System.out.println(x: "Welcome to Java Programming");
    }
}
```

```java
}

public class AccessM {
    Run | Debug
    public static void main(String[] args) {

        // Creating object
        Student s1 = new Student(name: "Nilesh", rollNo: 101);

        // Calling public method
        s1.displayDetails();

        // Calling protected method
        s1.checkResult(marks: 75);

        // Calling default method
        s1.showMessage();

        // Loop Example
        System.out.println(x: "Numbers from 1 to 5:");
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

Name:-Ragav Sharma

PRN:-202401110012